

Problem 1 - Washing Machine Mount Design (1 DOF)

Problem 1a

The Matlab code for this problem is listed in Appendix 1.

Figure 1 shows the load force produced by the 1 kg imbalanced mass at a 0.5 m radius from the center of the washing machine as a function of angular speed in RPM.

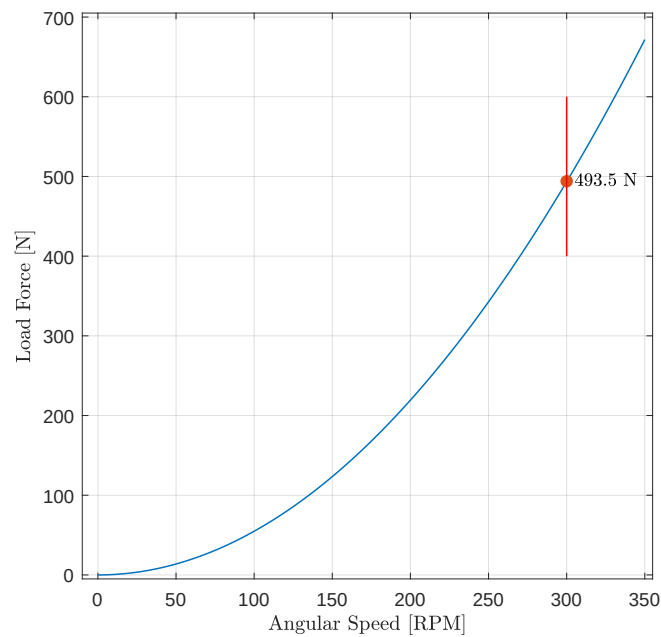


Figure 1: Load force of 1 kg imbalanced mass at a 0.5 m radius versus washing machine angular speed.

The force amplitude is 493.5 N at a frequency of 5 Hz when the washing machine operates at 300 RPM.

Problem 1b

The static displacement of the load in the washing machine must be less than 1 cm (i.e., the deflection is calculated with only the 10 kg load mass). The corresponding mount stiffness, k_s , is **9,810 $\frac{\text{N}}{\text{m}}$** .

The natural frequency, ω_o , of the mount is **1.5 Hz** (or $9.4 \frac{\text{radians}}{\text{s}}$).

Problem 1c

Figure 2 shows the transmission loss of the system for a set of damping ratios ranging from 0.001 to 1.

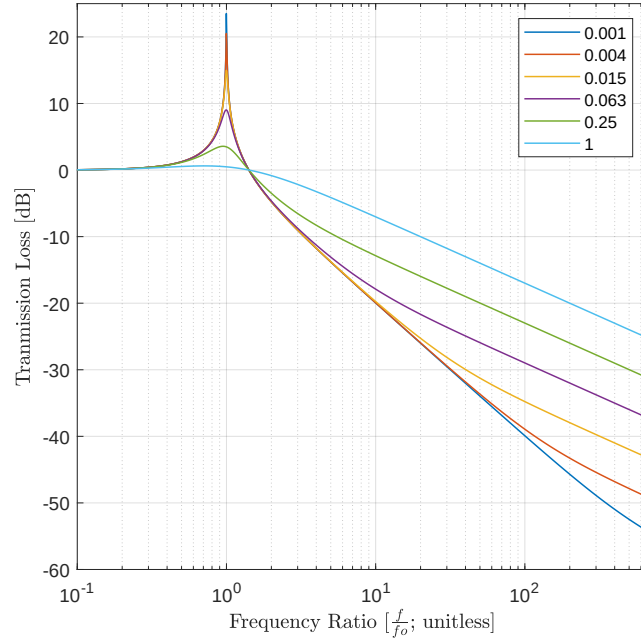


Figure 2: Transmission loss of the system. Its natural frequency is 1.5 Hz.

Problem 1d

Figure 3 shows the force applied to the foundation as a function of RPM.

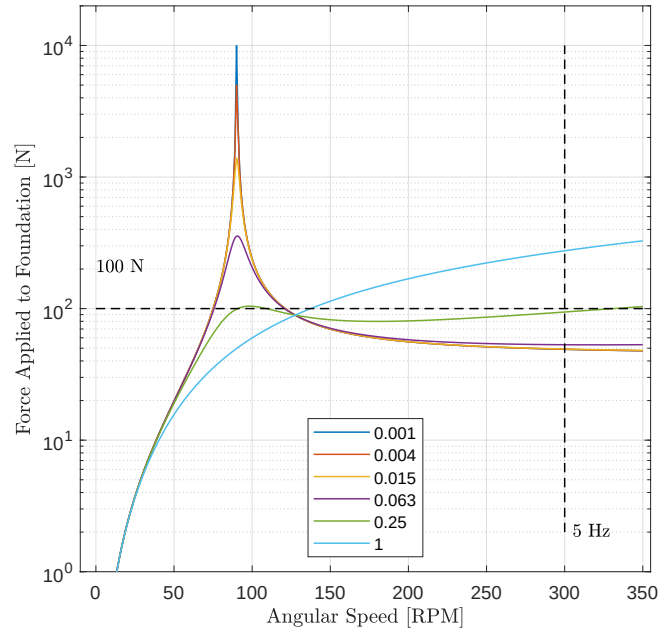


Figure 3: Force applied to the foundation versus RPM.

There is no damping ratio for which the applied force is less than 100 N across the whole operating range. The best damping ratio is $\zeta = 0.25$. With this damping ratio, the force applied to the foundation still

exceeds the 100 N target in two regions: 91 to 109 RPM and 331 to 350 RPM. In both regions the force is about 104 N.

Problem 1e

Figure 4 shows the admittance as a function of RPM.

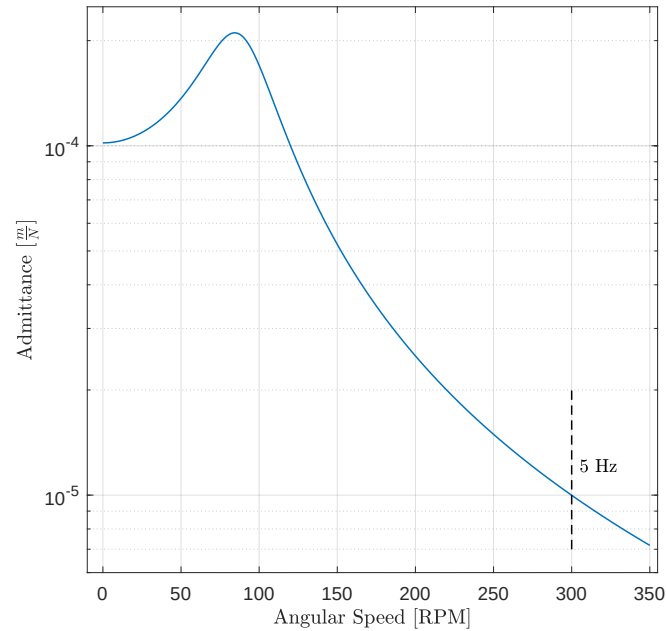


Figure 4: Admittance versus RPM.

The admittance at 300 RPM is $1e-5 \frac{m}{N}$.

Problem 1f

Figure 5 shows the displacement of the washing machine as a function of RPM.

The displacement of the washing machine at 300 RPM is 5×10^{-3} m. This amount of displacement for the 300 RPM operating state seems excessive. The washing machine undergoes larger momentary displacements as it spins up to the 300 RPM operating state, in particular around 100 RPM.

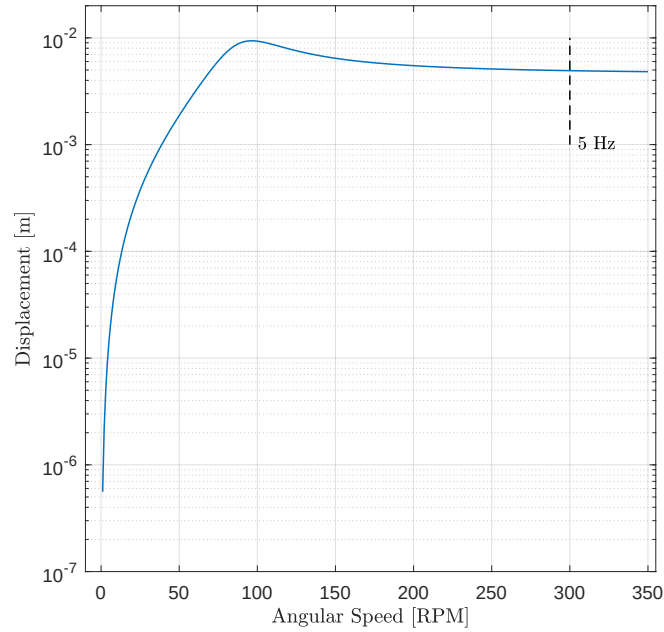


Figure 5: Displacement versus RPM.

Problem 2 - Washing Machine Mount Design (2 DOF)

Problem 2a

The Matlab code for this problem is listed in Appendix 2.

Figure 1 illustrates the two-stage vibration mount considered here (see slide 8 of 42 for Lecture 14 on Monday, March 3, 2025). Modeling will be done using discrete viscous damping elements.

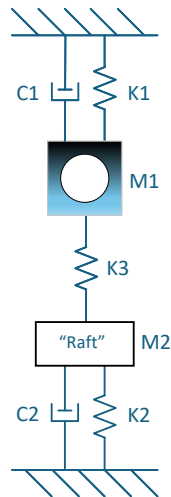


Figure 1: Two-stage vibration mount system for the washing machine.

Problem 2b

Table 1 lists the initial stiffness and damping coefficients for the system.

M1	110	kg
K1	100	$\frac{\text{N}}{\text{m}}$
C1	5	$\frac{\text{kg}}{\text{s}}$

M2	100	kg
K2	100	$\frac{\text{N}}{\text{m}}$
C2	5	$\frac{\text{kg}}{\text{s}}$

M3	0	kg
K3	1,000	$\frac{\text{N}}{\text{m}}$
C3	5	$\frac{\text{kg}}{\text{s}}$

Table 1: Initial values of stiffness and damping coefficients.

Problem 2c

Figure 2 illustrates the admittance of the washing machine and the raft with the force applied to the washing machine.

At the operating speed of 300 RPM, the admittance of the washing machine is $9.31 \times 10^{-6} \frac{\text{m}}{\text{N}}$.

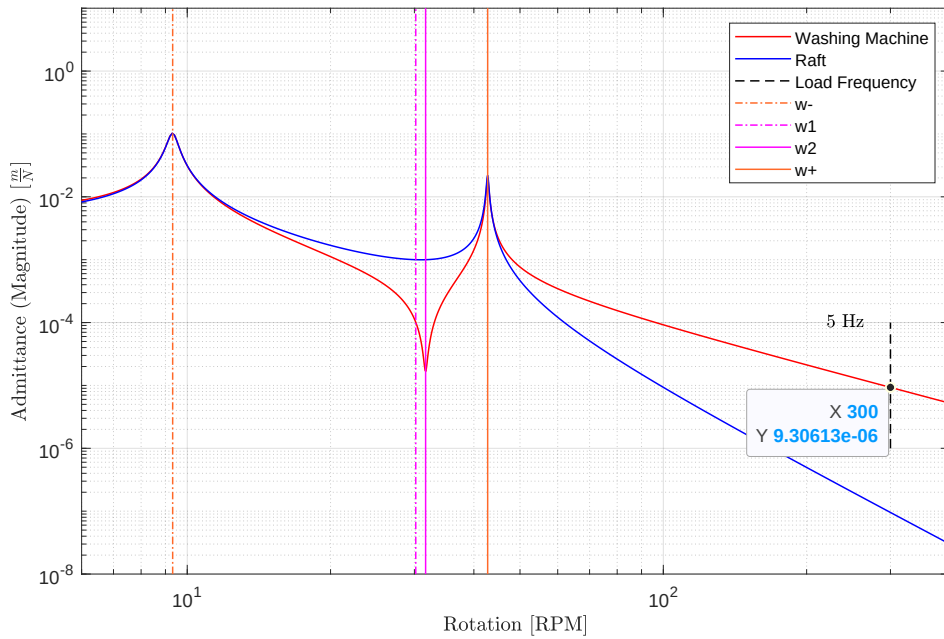


Figure 2: Admittance of the washing machine and the raft with a force on the washing machine.

Problem 2d

Figure 3 illustrates the displacement of the washing machine and the raft with the force applied to the washing machine.

At the operating speed of 300 RPM, the displacement of the washing machine is $4.6 \times 10^{-3} \text{ m}$, which is less than the $5.0 \times 10^{-3} \text{ m}$ displacement with the first-order system.

The displacement of the raft is $4.7 \times 10^{-5} \text{ m}$.

Problem 2e

Figure 4 illustrates the force applied to the ground by the raft. This is calculated by multiplying the displacement of the raft by the K_2 spring constant.

The load on the ground is always less than 100 N.

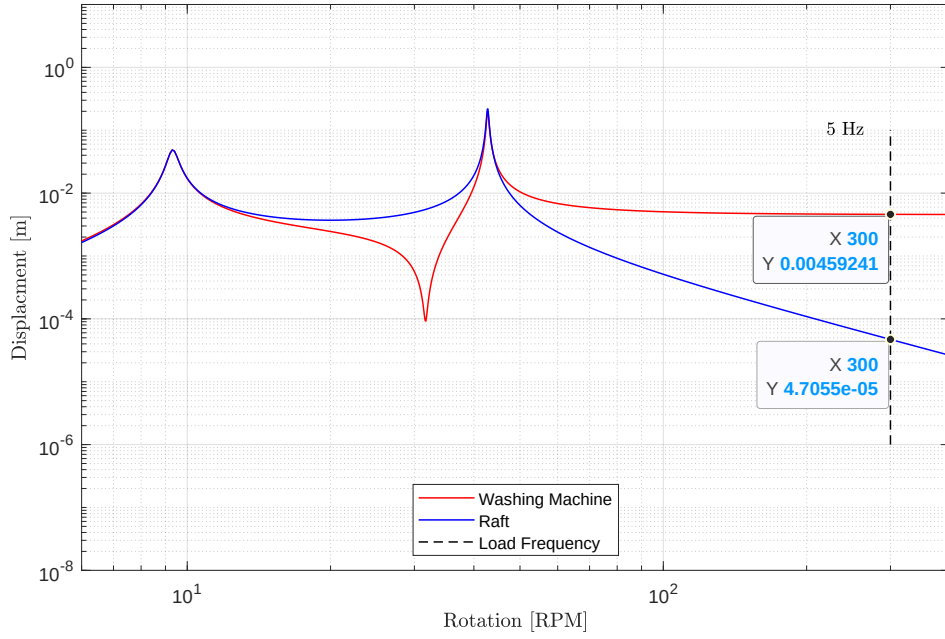


Figure 3: Displacement of the washing machine and the raft with a force on the washing machine.

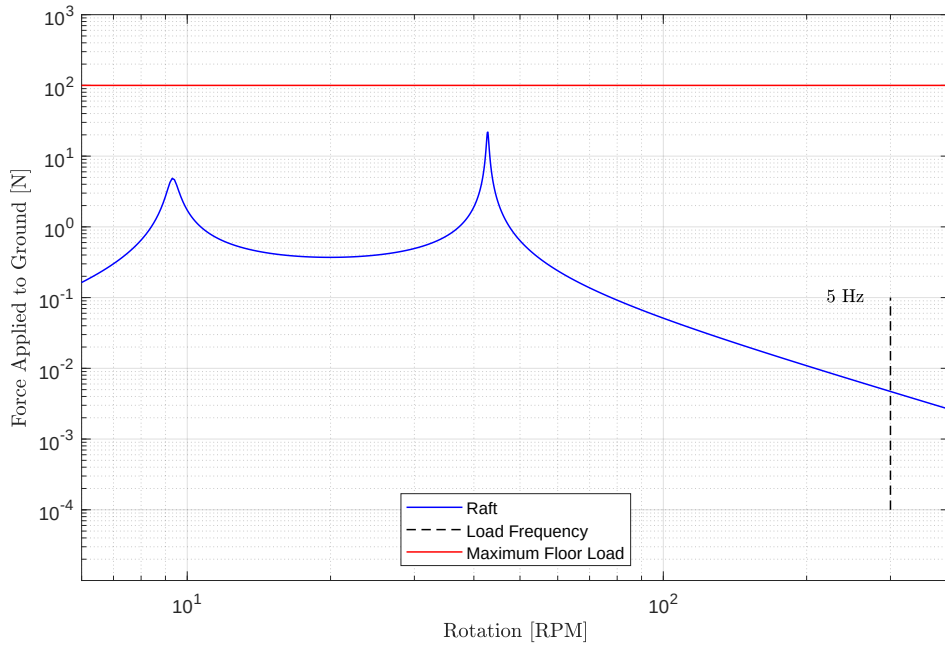


Figure 4: Force applied to the ground by the raft.

Problem 2f

The initial parameter set chosen, see Table 1, facilitated a displacement of the washing machine of 4.6×10^{-3} m at 300 RPM, which is less than the 5.0×10^{-3} m found for the 1 DOF system. The force applied to the ground by the raft is less than the required 100 N maximum.

Problem 2g

The initial parameter set chosen, see Table 1, facilitated a displacement of the washing machine of 4.6×10^{-3} m at 300 RPM, which is less than the 5.0×10^{-3} m found for the 1 DOF system. The force applied to the ground by the raft is less than the required 100 N maximum.

Problem 3 - Washing Machine Dynamic Vibration Absorber Design

The Matlab code for this problem is listed in Appendix 3.

Part 3a

Figure 1 illustrates the dynamic vibration absorber (DVA) system.

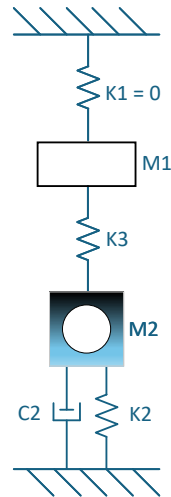


Figure 1: Dynamic vibration absorber system for the washing machine.

Part 3b

The Matlab code for this problem is listed in Appendix 3.

A DVA is used at a particular operating frequency, here 300 RPM or 5 Hz. Not used across a wide range of operating RPMs.

The ramp up to 300 RPM occurs quickly so that the large displacements are transient.

Higher roll-off of admittance with a DVA system, relative to the system in Problem 1 (see video at -09:35 mark).

Adding damping to the DVA system reduces the prominence of its two resonance peaks, while still reducing the amount of admittance and displacement of the original mass.

Figure 2 ph

ph

Part 3c

Figure 3 ph

ph

Part 3c

Figure 4 ph

ph

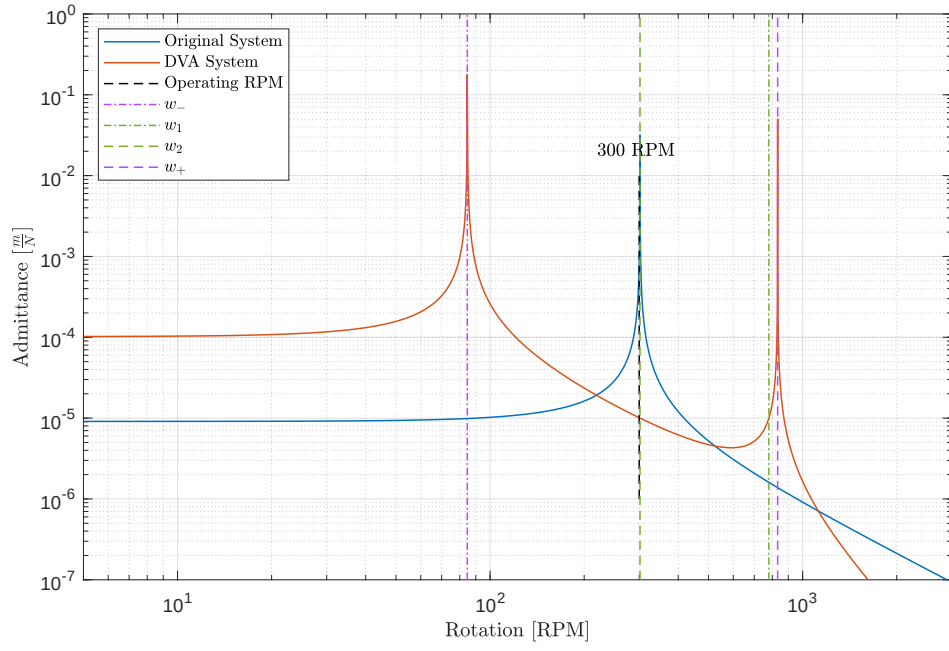


Figure 2: Equations from DVA tutorial lecture.

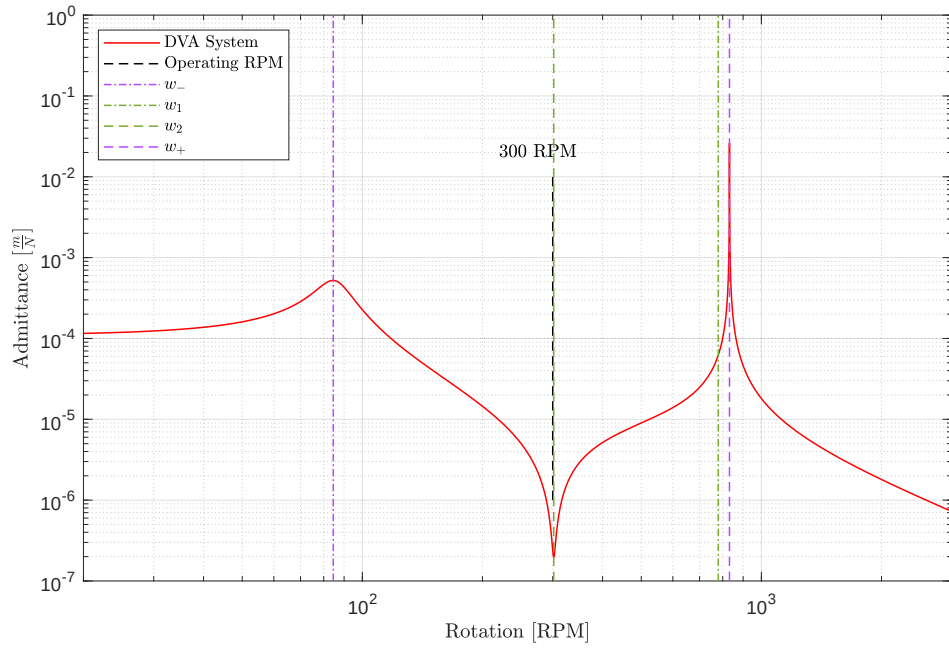


Figure 3: Solution using nDOF script-file function.

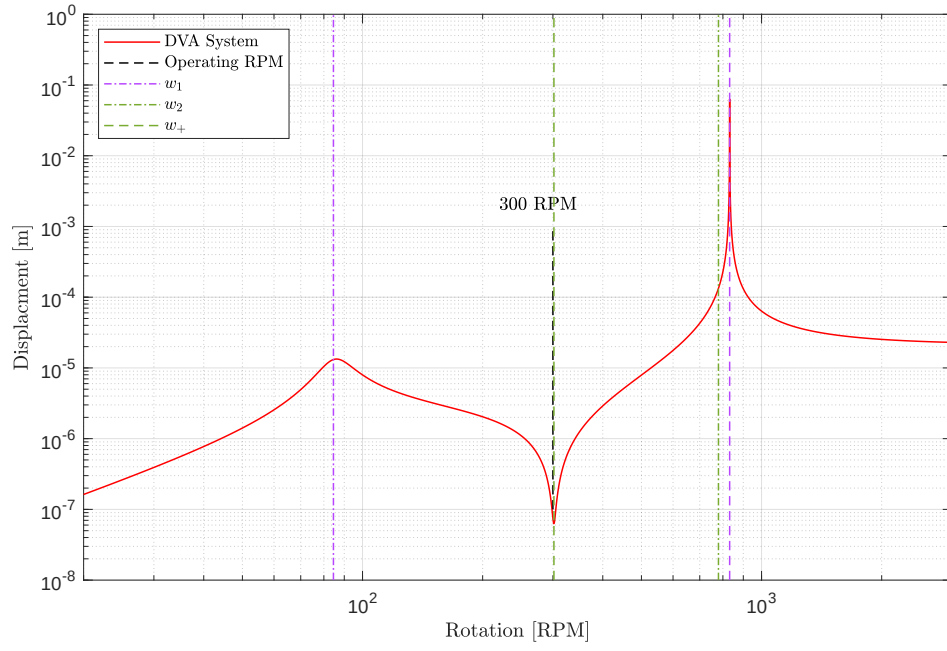


Figure 4: Displacement of the washing machine.

Part 3d

Figure 5 ph
ph

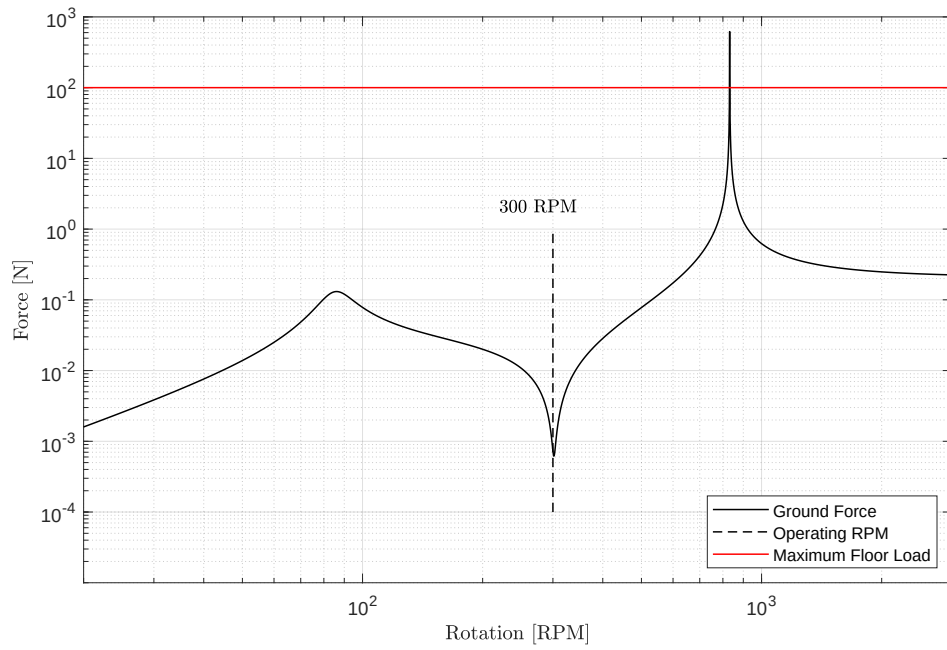


Figure 5: Force applied to the floor.

Problem 4 - Washing Machine Abuse Testing

The Matlab code for this problem is listed in Appendix [4](#).

Part 3a

1 Appendix - Matlab Code for Problem 1

```
1
2
3
4 %% Synopsis
5
6 % Problem 1 – Washing Machine Mount Design – 1 Degree of Freedom (DOF)
7
8
9
10 %% Notes
11
12 % The mass of the washing machine "does not go away."
13
14
15
16 %% Environment
17
18 close all; clear; clc;
19 % restoredefaultpath;
20
21 addpath( genpath( '../00 Support' ), '-begin' );
22
23 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
24 set( 0, 'DefaultFigureWindowSize', 'normal' );
25 set( 0, 'DefaultLineLineWidth', 0.8 );
26 set( 0, 'DefaultTextInterpreter', 'Latex' );
27
28 format LongG;
29
30 pause( 1 );
31
32
33
34 %% Anonymous Function Definitions
35
36 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
    load_distance;
37
38
39
40 %% Washing Machine Information
41
42 machine.mass = 100; % kg
43 machine.rotation_speed = 31.4; % radians\s or 300 rotations per minute (RPM)
44 machine.load_mass = 10; % kg
45 machine.static_displacement = 1e-2; % m
46
47
48
49 %% Load Force Information
50
51 dynamic_force.mass = 1; % kg
52 dynamic_force.radius = 0.5; % m
53
54 % Maximum force applied to foundation is 100 N.
55
56
57
58 %% Part 1a
59
60 rpm = 0:1:350;
61 rpm_conversion_to_radians_per_second = 0.10472;
62 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians\s
63
64
65 % Characteristics of the dynamic load.
66 force_frequency = 300 * rpm_conversion_to_radians_per_second / ( 2 * pi ); % 5 Hz
67 h_force( dynamic_force.mass, 300*rpm_conversion_to_radians_per_second, dynamic_force.radius )
    ; % 493.5 N
68
69
70 figure( 'Name', 'Problem 1a – Load Force Versus Angular Velocity' ); ...
71 plot( rpm, h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ) ); hold on;
72 plot( 300, 494, 'Marker', '.', 'MarkerSize', 20 );
73 line( [ 300, 300 ], [ 400 600 ], 'Color', 'r' );
```

```

74     text( 305, 494, '493.5 N' ); grid on;
75     xlabel( 'Angular Speed [RPM]' ); ylabel( 'Load Force [N]' );
76     axis( [ -10 355 -5 705 ] );
77
78
79
80 %% Part 1b
81
82 maximum_allowable_displacement = 1e-2; % m
83 ks = ( machine.load_mass * 9.81 ) / maximum_allowable_displacement; % 9,810 N/m
84
85
86 total_mass = machine.mass + machine.load_mass; % 110 kg
87 wo = sqrt( ks / total_mass ); % 9.4 radians/s
88 fo = wo / (2*pi); % 1.5 Hz - Natural frequency of the mount.
89
90
91
92 %% Part 1c
93
94 frequency_set = 0.1:0.01:1e3;
95 r = frequency_set ./ fo;
96
97 damping_ratio = logspace( log10(0.001), log10(1), 6 );
98
99 h_transmissibility = @( epsilon, r ) sqrt( ( 1 + (2.*epsilon.*r).^2 ) ./ ( ( 1 - r.^2 ).^2 + (
100     2.*epsilon.*r ).^2 ) );
101
102 figure( 'Name', 'Problem 1c - Transmission Loss' ); ...
103     hold on;
104     %
105     for index = 1:numel( damping_ratio )
106         plot( r, 10*log10( h_transmissibility( damping_ratio( index ), r ) ) );
107     end
108     %
109     xlabel( 'Frequency Ratio [ $\frac{f}{f_0}$ ; unitless]' ); ylabel( 'Transmission Loss [dB]' );
110     legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'NorthEast' );
111     axis( [ 0.1 665 -60 25 ] );
112     grid on; box on;
113     set( gca, 'XScale', 'log' );
114     %
115     % set((gcf, 'units', 'point', 'pos', [ 200 200 493*0.8 744*0.5 ] );
116     % pos = get( gcf, 'Position' );
117     % set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [ pos(3)
118     %     , pos(4) ] );
119     % if ( PRINT_FIGURES == 1 )
120     %     print(gcf, 'Homework_3_Part_1c', '-dpdf', '-r0' );
121     % end
122
123
124 %% Part 1d
125
126 rpm = 0:1:350; % rotations per minute
127 rpm_conversion_to_radians_per_second = 0.10472; % radians/s
128 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians/s
129
130 frequency_set = angular_velocity / (2*pi);
131 r = frequency_set ./ fo;
132
133
134 figure( 'Name', 'Problem 1d - Applied Force to Foundation' ); ...
135     hold on;
136     %
137     for index = 1:numel( damping_ratio )
138         plot( rpm, h_transmissibility( damping_ratio( index ), r ) .* h_force( dynamic_force.mass
139             , angular_velocity, dynamic_force.radius ) );
140     end
141     %
142     line( [ 300, 300 ], [ 2 1e4 ], 'Color', 'k', 'LineStyle', '—' );
143     text( 305, 2, '5 Hz' ); grid on;
144     line( [ 0 350 ], [ 100 100 ], 'Color', 'k', 'LineStyle', '—' );
145     text( 0, 205, '100 N' ); grid on;
146     xlabel( 'Angular Speed [RPM]' ); ylabel( 'Force Applied to Foundation [N]' );
147     legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'South' );
148     axis( [ -10 355 1 2e4 ] );
149     grid on; box on;

```

```

149     set( gca, 'YScale', 'log' );
150     %
151     % set( gcf, 'units', 'point', 'pos', [ 200 200      493*0.8 744*0.5 ] );
152     %     pos = get( gcf, 'Position' );
153     %     set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
154         %         , pos(4)] );
155     %         if ( PRINT_FIGURES == 1 )
156     %             print(gcf, 'Homework_3_Part_1d', '-dpdf', '-r0' );
157     %         end
158
159
160 %% Part 1e
161
162 % The best damping ratio is 0.25.
163
164 C = 0.25 * 2*sqrt( ks * total_mass ); % 519.4 kg\s
165
166 % The units of a viscous damping coefficient are Newton-seconds per meter (Ns/m) or kilograms per
167     % second (kg/s).
168
169 m = total_mass;
170 k = [ ks 0 ];
171 dampings = [ C 0 ];
172 admittance = nDOF_direct_solution( m, k, dampings, frequency_set, 'admittance' );
173
174
175 figure( 'Name', 'Problem 1e - Admittance' ); ...
176 semilogy( rpm, abs( admittance ) ); grid on;
177 xlabel( 'Angular Speed [RPM]' ); ylabel( 'Admittance  $[\frac{m}{N}]$ ' );
178 set( gca, 'YScale', 'log' );
179 %
180 line( [ 300, 300 ], [ 7e-6 2e-5 ], 'Color', 'k', 'LineStyle', '—' );
181 text( 305, 1.2e-5, '5 Hz' ); grid on;
182 axis( [ -10 355 6e-6 2.5e-4 ] );
183 %
184 % set( gcf, 'units', 'point', 'pos', [ 200 200      493*0.8 744*0.5 ] );
185 %     pos = get( gcf, 'Position' );
186 %     set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
187         %         , pos(4)] );
188 %         if ( PRINT_FIGURES == 1 )
189 %             print(gcf, 'Homework_3_Part_1e', '-dpdf', '-r0' );
190 %         end
191
192 % return
193
194 %% Part 1f
195 displacement = abs( admittance ) .* h_force( dynamic_force.mass, angular_velocity, dynamic_force.
196     radius );
197
198 figure( 'Name', 'Displacement' ); ...
199 plot( rpm, displacement ); grid on;
200 xlabel( 'Angular Speed [RPM]' ); ylabel( 'Displacement [m]' );
201 line( [ 300, 300 ], [ 1e-3 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
202 text( 305, 1e-3, '5 Hz' ); grid on;
203 set( gca, 'YScale', 'log' );
204 axis( [ -10 355 1e-7 2e-2 ] );
205 %
206 % set( gcf, 'units', 'point', 'pos', [ 200 200      493*0.8 744*0.5 ] );
207 %     pos = get( gcf, 'Position' );
208 %     set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
209         %         , pos(4)] );
210 %         if ( PRINT_FIGURES == 1 )
211 %             print(gcf, 'Homework_3_Part_1f', '-dpdf', '-r0' );
212 %         end
213
214
215 %% Clean-up
216
217 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
218     monitors = get( 0, 'MonitorPositions' );
219     if ( size( monitors, 1 ) == 1 )
220         autoArrangeFigures( 3, 4, 1 );
221     elseif ( 1 < size( monitors, 1 ) )

```

```

222         autoArrangeFigures( 2, 2, 2 );
223     end
224 end
225
226 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
227
228
229
230 %% Reference(s)
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245 %% Displacement Versus Frequency Ratio
246 %
247 % Fo = 1;
248 % wf = 4; % Force frequency, radians/s
249 %
250 % lambda = 1;
251 % k = (2*pi)/lambda;
252 %
253 % phase_offset = 0;
254 %
255 % w = 5;
256 % r = wf / w;
257 %
258 % time_indices = 0:1e-2:5;
259 %
260 % h_x = @( Fo, k, wf, time_indices, phase_offset, r, epsilon ) ( Fo./k.*sin(4*time_indices -
    phase_offset) ) ./ ( sqrt( (1 - r^2)^2 + (2*r*epsilon)^2 ) );
261 %
262 % figure( ); ...
263 %     epsilon = 0;
264 %     % plot( time_indices, h_x( Fo, k, wf, time_indices, phase_offset, r, epsilon ) ); hold
    on;
265 %
266 %     for wf = 10:-1:1
267 %         wf
268 %         epsilon = 0.25;
269 %         r = wf / w;
270 %         plot( h_x( Fo, k, wf, time_indices, phase_offset, r, epsilon ) ); hold on;
271 %         keyboard
272 %     end
273 %
274 %     legend( '', '' );
275 %     xlabel( 'Frequency Ratio [WU]' ); ylabel( 'Normalized Maximum Displacement [WU]' );

```

2 Appendix - Matlab Code for Problem 2

```
1
2
3
4 %% Synopsis
5
6 % Problem 2 — Washing Machine Two-stage Mount Design
7
8 % See Lecture 14, Monday, March 3, 2025
9
10
11
12 %% Note(s)
13
14 % A 2 degree-of-freedom (DOF) system will provide isolation performance 1 DOF system.
15
16 % Allowing the top and bottom masses, the washing machine and the raft ,
17 % respectively , to each be attached to ground independently.
18
19 % With the force applied to the washing machine and the displacement of the
20 % washing machine, use FSF( :, 1, 1 ).
21
22 % However, if the force is applied elsewhere, than FSF( :, 2, 2, ) might
23 % need to be used.
24
25
26
27 %% Environment
28
29 close all; clear; clc;
30 % restoredefaultpath;
31
32 addpath( genpath( './00 Support' ), '-begin' );
33
34 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
35 set( 0, 'DefaultFigureWindowStyle', 'normal' );
36 set( 0, 'DefaultLineLineWidth', 0.8 );
37 set( 0, 'DefaultTextInterpreter', 'Latex' );
38
39 format ShortG;
40
41 pause( 1 );
42
43
44
45 %% Define Anonymous Functions
46
47 h_f_to_rpm = @( f ) ( f*2*pi ) / 0.10471;
48 h_w_to_rpm = @( w ) h_f_to_rpm( w*2*pi );
49 h_w_to_f = @( w ) w/(2*pi);
50 h_rpm_to_f = @( rpm ) ( rpm * 0.10471 ) / ( 2 * pi );
51
52 % return
53
54 %% Problem 2a
55
56 % See report .
57
58
59
60 %% Problem 2b — Blocked Frequencies of Resonance
61
62 % The frequency of the forcing function is 5 Hz and the amplitude is 493.5 N.
63 fo = 5; % Hz
64 wo = 2*pi*fo; % 31.4 radians\s
65
66
67 m1 = 100 + 10; % kg — Total mass of the washing machine and the load.
68 k1 = 1e2; % N\m
69 c1 = 5;
70
71 m2 = 100; % kg — UPPER LIMIT OF 100 kg
72 k2 = 1e2; % N\m
73 c2 = 5;
74
75 k3 = 1e3; % Admittance: 9.31e-6 m\N
```



```

76 %
77 % Note(s):
78 %
79 % a.) Strong coupling produces a high w+.
80
81
82 w1 = sqrt( ( k1 + k3 ) / m1 ); % radians/s; washing machine
83 f1 = w1 / (2*pi);
84 rpm1 = f1*(2*pi)/0.10471;
85
86 w2 = sqrt( ( k2 + k3 ) / m2 ); % radians/s; raft
87 f2 = w2 / (2*pi);
88 rpm2 = f2*(2*pi)/0.10471;
89
90 % Note: If k3 is set to zero, then these frequencies represent the
91 % resonance frequencies for each mass on their own.
92
93
94
95 %% Problem 2b – Coupled Frequencies
96
97 mu_4 = ( k3^2 / (m1*m2) ); % (radians/s)^4
98 mu = ( mu_4 )^0.25;
99
100 w(1) = 0.5*( ( w1^2 + w2^2 ) + sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
101 w(2) = 0.5*( ( w1^2 + w2^2 ) - sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
102 w = sqrt( w );
103
104
105 w_minus = w(2); % 0.97 radians/s (lower than w1, 1, and w2, 1.05 radians/s)
106 f_minus = w(2)/(2*pi); % 0.1545 Hz
107 rpm_minus = h_f_to_rpm( f_minus );
108 %
109 w_plus = w(1); % 1.08 radians/s (higher than w1, 1, and w2, 1.05 radians/s)
110 f_plus = w(1)/(2*pi); % 0.17124 Hz
111 rpm_plus = h_f_to_rpm( f_plus );
112 %
113 % Note(s):
114 %
115 % The blocked frequencies, w1 and w2, are always between these two frequencies.
116
117
118 clear w mu_4;
119
120
121
122 %% Problem 2b – Mode Shapes
123
124 phi_plus = [ ...
125 1; ...
126 -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
127 ];
128
129 phi_minus = [ ...
130 1; ...
131 (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
132 ];
133
134
135 % Note(s):
136 %
137 % When m1 = m2 and k1 = k3,
138 %
139 % a.) At w_minus, m1 and m2 move the same amount and are in-phase.
140 % b.) At w_plus, m1 and m2 move the same amount, but are out-of-phase.
141
142 % In general, for a 2 DOF system there will be 2 modes (in-phase and out-of-phase).
143
144
145 % With different initial conditions (no forcing), the behaviour is a weighted sum of the two
146 % modes.
147
148
149 %% Regions of Interest
150
151 % 5 regions of interest:
152

```

```

153 % 1.) Low frequency:  $w < w_{\text{minus}}$ :
154 %
155 % Stiffness controlled; two masses moving together; strong coupling ( $k_3$  high).
156 %
157 %
158 % 2.) Force at a resonance frequency:  $w = w_{\text{minus}}$  (lowest resonance frequency):
159 %
160 % At lowest resonance frequency; at  $w_{\text{minus}}$  mode, masses moving in same direction (in-phase)
    with same high amplitude
161 %
162 %
163 % 3.) Forcing frequency between  $w_{\text{minus}}$  and  $w_2$ 
164 %
165 %
166 % 4.) Forcing frequency  $w = w_2$ 
167 %
168 %
169 % 5.) Forcing frequency  $w < w < w_+$ 
170 %
171 % No a special frequency.
172 %
173 %
174 % 6.) Forcing frequency  $w = w_+$ 
175 %
176 % At highest resonance frequency; at  $w_{\text{plus}}$  mode, masses moving in opposite directions (out-of-
    phase) with same high amplitude
177 %
178 %
179 % 7.) Forcing frequency  $w_+ < w$ 
180 %
181 % Mass controlled region; mass 1 (washing machine) moves more than mass 2 (raft);
182 %
183 %
184 %
185 %% Problem 2c
186 %
187 masses = [ m1 m2 ];
188 stiffnesses = [ k1 k3 k2 ];
189 dampings = [ c1 0 c2 ];
190 %
191 rpm = 0:0.1:500; % rotations per minute
192 rpm_conversion_to_radians_per_second = 0.10472;
193 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians\s
194 frequencies = angular_velocity / (2*pi); % Hz
195 %
196 FRF = nDOF_direct_solution( masses, stiffnesses, dampings, frequencies, 'admittance' );
197 %
198 %
199 admittance = zeros( numel( frequencies ), 1 );
200 %
201 for index = 1:1:numel( frequencies )
202     temp = diag( squeeze( FRF( index, 1, 1 ) ) ); % Check indexing to ensure force on washing
        machine.
203     admittance( index ) = abs( temp );
204 end
205 %
206 clear temp temp2;
207 %
208 washing_machine.admittance = admittance;
209 %
210 %
211 admittance = zeros( numel( frequencies ), 1 );
212 %
213 for index = 1:1:numel( frequencies )
214     temp = diag( squeeze( FRF( index, 1, 2 ) ) ); % Check indexing to ensure force on washing
        machine.
215     admittance( index ) = abs( temp );
216 end
217 %
218 clear temp temp2;
219 %
220 raft.admittance = admittance;
221 %
222 %
223 figure( 'Name', 'Admittance - Magnitude' ); ...
224 h1 = loglog( rpm, washing_machine.admittance, 'Color', 'r' ); hold on;
225 h2 = loglog( rpm, raft.admittance, 'Color', 'b' );
226 h3 = line( [ 300 300 ], [ 1e-6 1e-4 ], 'Color', 'k', 'LineStyle', '—' ); grid on;

```

```

227     text( 220, 1e-4, '5 Hz' );
228 %
229 h4 = line( [ rpm_minus rpm_minus ], [ 1e-8 1e1 ], 'Color', [ 1.00, 0.41, 0.16 ], 'LineStyle',
230     '—' );
231 %
232 h5 = line( [ rpm1 rpm1 ], [ 1e-8 1e1 ], 'Color', 'm', 'LineStyle', '—' );
233 %
234 h6 = line( [ rpm2 rpm2 ], [ 1e-8 1e1 ], 'Color', 'm', 'LineStyle', '—' );
235 %
236 h7 = line( [ rpm_plus rpm_plus ], [ 1e-8 1e1 ], 'Color', [ 1.00, 0.41, 0.16 ], 'LineStyle',
237     '—' );
238 %
239 legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Washing Machine', 'Raft', 'Load Frequency', 'w—', 'w1', 'w2', 'w+', 'Location', 'NorthEast' );
240 %
241 xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance (Magnitude) [ $\frac{m}{N}$ ]' );
242 axis( [ 6 400 1e-8 1e1 ] );
243 %
244 % set( gcf, 'units', 'point', 'pos', [ 200 200 493*0.8 744*0.5 ] );
245 % pos = get( gcf, 'Position' );
246 % set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [ pos(3)
247     , pos(4) ] );
248 % if ( PRINT_FIGURES == 1 )
249 %     print(gcf, 'Homework_3_Part_2c', '-dpdf', '-r0' );
250 % end
251
252 %% Problem 2d
253
254 dynamic_force.mass = 1; % kg
255 dynamic_force.radius = 0.5; % m
256
257 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
258     load_distance;
259
260 temp = h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).';
261
262 figure( 'Name', '' ); ...
263 h1 = loglog( rpm, washing_machine.admittance.*temp, 'Color', 'r' ); hold on;
264 h2 = loglog( rpm, raft.admittance.*temp, 'Color', 'b' );
265 h3 = line( [ 300 300 ], [ 1e-6 1e-1 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
266 text( 220, 1e-1, '5 Hz' );
267 legend( [ h1 h2 h3 ], 'Washing Machine', 'Raft', 'Load Frequency', 'Location', 'South' );
268 xlabel( 'Rotation [RPM]' ); ylabel( 'Displacement [m]' );
269 axis( [ 6 400 1e-8 1e1 ] );
270 %
271 % set( gcf, 'units', 'point', 'pos', [ 200 200 493*0.8 744*0.5 ] );
272 % pos = get( gcf, 'Position' );
273 % set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [ pos(3)
274     , pos(4) ] );
275 % if ( PRINT_FIGURES == 1 )
276 %     print(gcf, 'Homework_3_Part_2d', '-dpdf', '-r0' );
277 % end
278
279 %% Problem 2e
280
281 % Displacement of the raft multiplied by the K2 spring constant.
282
283 dynamic_force.mass = 1; % kg
284 dynamic_force.radius = 0.5; % m
285
286 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
287     load_distance;
288
289 temp = h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).';
290
291 figure( 'Name', 'Force Applied to Ground by Raft' ); ...
292 h1 = loglog( rpm, raft.admittance.*temp*k2, 'Color', 'b' ); hold on;
293 h2 = line( [ 300 300 ], [ 1e-4 1e-1 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
294 h3 = line( [ 6 400 ], [ 100 100 ], 'Color', 'r' );
295 text( 220, 1e-1, '5 Hz' );
296 legend( [ h1 h2 h3 ], 'Raft', 'Load Frequency', 'Maximum Floor Load', 'Location', 'South' );
297 xlabel( 'Rotation [RPM]' ); ylabel( 'Force Applied to Ground [N]' );

```

```

297     axis( [ 6 400 1e-5 1e3 ] );
298     %
299     % set( gcf, 'units', 'point', 'pos', [ 200 200 493*0.8 744*0.5 ] );
300     % pos = get( gcf, 'Position' );
301     % set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [pos(3)
    , pos(4)] );
302     % if ( PRINT_FIGURES == 1 )
303     %     print(gcf, 'Homework_3_Part_2e', '-dpdf', '-r0' );
304     % end
305
306
307
308 %% Problem 2f
309
310 % return
311
312 %% Problem 2g
313
314 % return
315
316 %% 2 DOF Example
317
318 % masses = [ 50 1 ];
319 % stiffnesses = [ 100e3 1800 0 ];
320 %
321 % dampings = [ 0 0 0 ];
322 % dampings = [ 0 (1 + 0.1*1j) (1 + 0.1*1j) ];
323 %
324 % frequencies = 0:0.01:40;
325 %
326 % FRF = nDOF_direct_solution( masses, stiffnesses, dampings, frequencies, 'admittance' ); %
    4001-by-2-by-2
327 %
328 %
329 % admittance = zeros( numel( frequencies ), 1 );
330 %
331 % for index = 1:1:numel( frequencies )
332 %     temp = diag( squeeze( FRF( index, :, : ) ) );
333 %     temp2 = temp(1) + temp(2)*1j;
334 %     admittance( index ) = abs( temp2 );
335 % end
336 %
337 % clear temp1 temp2;
338 %
339 %
340 %% figure( ); ...
341 %% semilogy( frequencies, admittance ); grid on;
342 %% xlabel( 'Frequency [Hz]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
343 %
344 %
345 % figure( 'Name', 'Admittance' ); ...
346 % semilogy( frequencies, abs( FRF( :, 1, 1 ) ) ); hold on;
347 % semilogy( frequencies, abs( FRF( :, 1, 2 ) ) );
348 % semilogy( frequencies, abs( FRF( :, 2, 1 ) ) );
349 % semilogy( frequencies, abs( FRF( :, 2, 2 ) ) ); grid on;
350 % legend( 'M1 Movement with F1', 'M1 Movement with F2', 'M2 Movement with F1', 'M2
    Movement with F2', 'Location', 'North' );
351 % xlabel( 'Frequency [Hz]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
352 % axis( [ 5.5 10.5 1e-7 1e2 ] );
353
354
355 % figure( 'Name', 'Dynamic Stiffness (Reciprocal of Admittance)' ); ...
356 % semilogy( frequencies, 1 ./ abs( FRF( :, 1, 1 ) ) ); hold on;
357 % semilogy( frequencies, 1 ./ abs( FRF( :, 1, 2 ) ) );
358 % semilogy( frequencies, 1 ./ abs( FRF( :, 2, 1 ) ) );
359 % semilogy( frequencies, 1 ./ abs( FRF( :, 2, 2 ) ) ); grid on;
360 % legend( 'M1 Movement with F1', 'M1 Movement with F2', 'M2 Movement with F1', 'M2
    Movement with F2', 'Location', 'North' );
361 % xlabel( 'Frequency [Hz]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
362
363
364
365 %% Clean-up
366
367 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
368     monitors = get( 0, 'MonitorPositions' );
369     if ( size( monitors, 1 ) == 1 )
370         autoArrangeFigures( 3, 4, 1 );

```

```

371         elseif ( 1 < size( monitors , 1 ) )
372             autoArrangeFigures( 2, 2, 2 );
373         end
374     end
375
376     fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
377
378
379
380 %% Reference(s)
381
382 % https://tex.stackexchange.com/questions/295059/three-tables-side-by-side-on-one-page
383
384
385
386 %% Reference Code
387
388 % admittance = zeros( numel( frequencies ), 1 );
389 %
390 % for index = 1:1:numel( frequencies )
391 %     temp = diag( squeeze( FRF( index , 1, 1 ) ) ); % Check indexing to ensure force on washing
392 %         admittance( index ) = unwrap( angle ( temp ) ) * 180 / pi;
393 % end
394 % %
395 % clear temp temp2;
396 %
397 % washing_machine.admittance_phase = admittance;
398 %
399 %
400 % admittance = zeros( numel( frequencies ), 1 );
401 %
402 % for index = 1:1:numel( frequencies )
403 %     temp = diag( squeeze( FRF( index , 1, 2 ) ) ); % Check indexing to ensure force on washing
404 %         admittance( index ) = unwrap( angle ( temp ) ) * 180 / pi;
405 % end
406 % %
407 % clear temp temp2;
408 %
409 % raft.admittance_phase = admittance;
410 %
411 % figure( 'Name', 'Admittance - Phase' ); ...
412 %     h1 = semilogx( rpm, washing_machine.admittance_phase, 'Color', 'r' ); hold on;
413 %     h2 = semilogx( rpm, raft.admittance_phase , 'Color', 'b', 'LineStyle', '--' ); grid on;
414 %     h3 = line( [ 300 300 ], [ -200 200 ], 'Color', 'k', 'LineStyle', '--' ); grid on;
415 %     %
416 %     h4 = line( [ rpm_minus rpm_minus], [ -200 200 ], 'Color', 'g', 'LineStyle', '-.' );
417 %     %
418 %     line( [ rpm1 rpm1 ], [ -200 200 ], 'Color', 'm', 'LineStyle', '--' );
419 %     line( [ rpm2 rpm2 ], [ -200 200 ], 'Color', 'm', 'LineStyle', '--' );
420 %     %
421 %     h5 = line( [ rpm_plus rpm_plus ], [ -200 200 ], 'Color', 'g', 'LineStyle', '-' );
422 %     %
423 %     legend( [ h1 h2 h3 h4 h5 ], 'Washing Machine', 'Raft', 'Load Frequency', 'w-', 'w+', '
Location', 'NorthEast' );
424 %
425 %     xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
426 %     xlim( [ 6 400 ] );
427 %     % axis( [ 6 400 1e-8 1e1 ] );
428 %
429 %     set( gcf, 'units', 'point', 'pos', [ 200 200 493*0.8 744*0.5 ] );
430 %     % pos = get( gcf, 'Position' );
431 %     % set( gcf, 'PaperPositionMode', 'Auto', 'PaperUnits', 'points', 'PaperSize', [ pos
(3), pos(4) ] );
432 %
433 %     if ( PRINT_FIGURES == 1 )
434 %         print(gcf, 'Homework_3_Part_2c', '-dpdf', '-r0' );

```

3 Appendix - Matlab Code for Problem 3

```
1
2
3
4 %% Synopsis
5
6 % Problem 3 – Washing Machine Dynamic Vibration Absorber (DVA) Design
7
8
9
10 %% Note(s)
11
12 % At 300 RPM, the load frequency, this system will isolated vibration more
13 % than the 2 DOF and 1 DOF systems.
14
15 % The DVA behaves like a Helmholtz resonator, working as a mechanical notch
16 % filter at a particular frequency.
17
18 % DVAs work well in systems that have one dominate mode. A good
19 % example of this is building sway and its compensation. Based on the
20 % design and construction of a building, it will typically have a single,
21 % dominate mode.
22
23 % DVA is useful if you can not get away from the resonance frequency; can
24 % not tune a 2 DOF system when the resonance frequency is away from the
25 % forcing frequency.
26
27 % DVA introduces two resonances for the original resonance. These new
28 % resonances will produce greater motion at the new resonance frequencies.
29
30
31
32 %% Environment
33
34 close all; clear; clc;
35 % restoredefaultpath;
36
37 addpath( genpath( './00 Support' ), '-begin' );
38
39 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
40 set( 0, 'DefaultFigureWindowSize', 'normal' );
41 set( 0, 'DefaultLineLineWidth', 0.8 );
42 set( 0, 'DefaultTextInterpreter', 'Latex' );
43
44 format ShortG;
45
46 pause( 1 );
47
48
49
50 %% Define Anonymous Functions
51
52 h_rpm_to_f = @( rpm ) ( rpm * 0.10471 ) / ( 2 * pi );
53 h_rpm_to_w = @( rpm ) ( rpm * 0.10471 );
54
55 h_f_to_rpm = @( f ) ( f*2*pi ) / 0.10471;
56
57 h_w_to_f = @( w ) w/(2*pi);
58 h_w_to_rpm = @( w ) h_f_to_rpm( w*2*pi );
59
60
61 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
        load_distance;
62
63
64
65 %% Washing Machine Information
66
67 machine.mass = 100; % kg
68 machine.rotation_speed = 31.4; % radians/s or 300 rotations per minute (RPM)
69 machine.load_mass = 10; % kg
70 machine.static_displacement = 1e-2; % m
71
72
73
74 %% Load Force Information
```

```

75
76 dynamic_force.mass = 1; % kg
77 dynamic_force.radius = 0.5; % m
78
79 % Maximum force applied to foundation is 100 N.
80
81
82
83 %% Dynamic Vibration Absorber (DVA)
84
85 maximum_allowable_displacement = 1e-2; % m
86 ks = ( machine.load_mass * 9.81) / maximum_allowable_displacement; % 9,810 N\m
87
88
89 m = 110; % kg; washing machine mass and load mass
90 k = ks; % N\m
91
92 wo = sqrt( k / m); % 9.4 radians\s
93 fo = wo/(2*pi); % 1.5 Hz
94
95
96 m_dva = 15; % kg
97 k_dva = 1e5; % N\m
98 % k_dva = 2e5; % N\m
99 % k_dva = 1800; % N\m -> 7.7095e-05; 2.4411e-05; 0.23947 -> f2 IS LESS THAN f1
100 % k_dva = 1000; % N\m -> 7.2494e-05; 2.2954e-05; 0.22518
101 % k_dva = 100; % N\m -> 6.8007e-05; 2.1533e-05; 0.21124
102 % k_dva = 10; % N\m -> 6.7593e-05; 2.1402e-05; 0.20995
103 % k_dva = 1; % N\m -> 6.7552e-05; 2.1389e-05; 0.20983
104
105 wo_dva = sqrt( k_dva / m_dva ); % 81.7 radians\s
106 fo_dva = wo_dva/(2*pi); % 13.0 Hz
107
108
109
110 %% Blocked Frequencies (Equation 9.1 of the Notes)
111
112 m1 = m_dva;
113 k1 = 0; % Spring above DVA mass; does not exist; set to zero.
114
115 m2 = m;
116 k2 = k;
117
118 k3 = k_dva;
119
120
121 w1 = sqrt( ( k1 + k3 ) / m1 ); % 81.6 radians\s; smaller than the system resonant frequency.
122 f1 = w1/(2*pi); % 1.3 Hz
123
124 w2 = sqrt( ( k2 + k3 ) / m2 ); % 31.6 radians\s; greater than the system resonant frequency.
125 f2 = w2/(2*pi); % 1.6 Hz
126
127 % return
128
129 %% Coupled Frequencies (Equation 9.15 of the Notes)
130
131 mu_4 = ( k3^2 / (m1*m2) ); % 1,963.6 (radians\s)^4
132 mu = ( mu_4 )^0.25; % 6.66 radians\s
133
134 w(1) = 0.5*( ( w1^2 + w2^2 ) + sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
135 w(2) = 0.5*( ( w1^2 + w2^2 ) - sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
136 w = sqrt( w );
137
138
139 w_minus = w(2); % 8.2 radians\s (lower than 42.4 and 45.1 radians\s)
140 f_minus = w(2)/(2*pi); % 1.31 Hz
141 %
142 w_plus = w(1); % 12.6 radians\s (higher than 42.4 and 45.1 radians\s)
143 f_plus = w(1)/(2*pi); % 2.0 Hz
144 %
145 % Note(s):
146 %
147 % The blocked frequencies are always between these two frequencies.
148
149 % return
150
151 %% Mode Shapes (Equations 9.18 and 9.19 of the Notes)
152

```

```

153 phi_plus = [ ...
154     1; ...
155     -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
156     ];
157
158 phi_minus = [ ...
159     1; ...
160     (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
161     ];
162
163
164
165 %% Plot (1) Admittance Using Equations from DVA Lecture
166
167 MAXIMUM_RPM = 3e3;
168
169 F0 = 1;
170
171 w = 0:0.01:h_rpm_to_w( MAXIMUM_RPM );
172 f = w/(2*pi);
173 rpm = h_f_to_rpm( f );
174
175
176 m = [ 15 110 ];
177 k = [ 0 k_dva k2.*(1+0.2*1i) ];
178
179 admittance_modified = F0 ./ ( m(1)*m(2) ) * k(2) ./ (( w.^2 - w_plus^2 ) .* ( w.^2 - w_minus^2 ));
180 admittance_original = F0 ./ m(2) * 1 ./ ( w2^2 - w.^2 );
181
182
183 figure( 'Name', 'Admittance - Equations from DVA Lecture' ); ...
184 h1 = loglog( rpm, abs( admittance_original ) ); hold on;
185 h2 = loglog( rpm, abs( admittance_modified ) );
186 h3 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
187 text( 220, 2e-2, '300 RPM' );
188 h4 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
189     0.27, 1.00 ], 'LineStyle', '—' );
190
191 h5 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
192     ], 'LineStyle', '—' );
193 h6 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
194     ], 'LineStyle', '—' );
195
196 h7 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
197     0.27, 1.00 ], 'LineStyle', '—' ); grid on;
198
199 axis( [ 5 3e3 1e-7 1 ] );
200 legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Original System', 'DVA System', 'Operating RPM', ...
201     '$w_-$', '$w_1$', ...
202     '$w_2$', '$w_+$', ...
203     'Location', 'NorthWest', 'Interpreter', 'Latex' );
204 xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
205
206 %% Plot (2) Admittance Using nDOF Function
207
208 % Damping can be added by using a dampings vector or appending a complex
209 % value to the respective spring stiffness. THESE ARE NOT INTERCHANGABLE.
210
211 rpm = 0:0.1:MAXIMUM_RPM;
212 f = h_rpm_to_f( rpm );
213 w = f ./ (2*pi);
214
215 FRF = nDOF_direct_solution( m, k, [ 0 0 0 ], f, 'admittance' ); % Symmetric matrix.
216 %
217 squeeze( FRF( numel( f ), :, : ) );
218 %
219 % (1, 1) - Force on M1 and its associated displacement.
220 % (2, 2) - Force on M2 and its associated displacement.
221 %
222 % (1, 2) - Force on M1 and the displacement of M2.
223 % (2, 1) - Force on M2 and the displacement on M1.
224 %
225 % These are interchangeable; the same result.
226

```



```

227 figure( 'Name', 'Admittance - nDOF Calculated' ); ...
228 h1 = loglog( h_f_to_rpm( f ), abs( FRF( :, 1, 1 ) ), 'Color', 'r' ); hold on;
229 h2 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
230 text( 220, 2e-2, '300 RPM' );
231 h3 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
0.27, 1.00 ], 'LineStyle', '-.' );
232
233 h4 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
], 'LineStyle', '-.' );
234 h5 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
], 'LineStyle', '—' );
235
236 h6 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
0.27, 1.00 ], 'LineStyle', '—' ); grid on;
237
238 axis( [ 2e1 MAXIMUM_RPM 1e-7 1 ] );
239 legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
240 '$w_{-}$', '$w_{1}$', ...
241 '$w_{2}$', '$w_{+}$', ...
242 'Location', 'NorthWest', 'Interpreter', 'Latex' );
243 xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance  $\frac{m}{N}$ ' );
244
245
246 temp2 = abs( FRF( :, 1, 1 ) );
247 temp2( 3001 )
248
249
250
251 %% Displacement
252
253 dynamic_force.mass = 1; % kg
254 dynamic_force.radius = 0.5; % m
255
256 % angular_velocity = f / (2*pi);
257 % rpm = h_f_to_rpm( f );
258
259 temp = h_force( dynamic_force.mass, w, dynamic_force.radius ).';
260
261
262 figure( 'Name', 'Displacement' ); ...
263 h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*temp, 'Color', 'r' ); hold on;
264 h2 = line( [ 300 300 ], [ 1e-7 1e-3 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
265 text( 220, 2e-3, '300 RPM' );
266 h4 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_minus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
0.27, 1.00 ], 'LineStyle', '-.' );
267
268 h5 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f1 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
], 'LineStyle', '-.' );
269 h6 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f2 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
], 'LineStyle', '—' );
270
271 h7 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_plus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
0.27, 1.00 ], 'LineStyle', '—' ); grid on;
272
273 legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
274 '$w_{-}$', '$w_{1}$', ...
275 '$w_{2}$', '$w_{+}$', ...
276 'Location', 'NorthWest', 'Interpreter', 'Latex' );
277
278 xlabel( 'Rotation [RPM]' ); ylabel( 'Displacement [m]' );
279 axis( [ 2e1 MAXIMUM_RPM 1e-8 1 ] );
280
281
282 temp2 = abs( FRF( :, 1, 1 ) ).*temp;
283 temp2( 3001 )
284
285 % return
286
287 %% Ground Force
288
289 dynamic_force.mass = 1; % kg
290 dynamic_force.radius = 0.5; % m
291
292
293 temp = h_force( dynamic_force.mass, w, dynamic_force.radius ).';
294
295 close all
296 figure( 'Name', 'Ground Force' ); ...

```

```

297     h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*temp.*k2, 'Color', 'k' ); hold on;
298     h2 = line( [ 300 300 ], [ 1e-4 1e-0 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
299     text( 220, 2e-0, '300 RPM' );
300     h3 = line( [ 6 MAXIMUM_RPM ], [ 100 100 ], 'Color', 'r' );
301     legend( [ h1 h2 h3 ], 'Ground Force', 'Operating RPM', 'Maximum Floor Load', 'Location',
302             'SouthEast' );
303     xlabel( 'Rotation [RPM]' ); ylabel( 'Force [N]' );
304     axis( [ 2e1 MAXIMUM_RPM 1e-5 1e3 ] );
305
306     temp2 = abs( FRF( :, 1, 1 ) ).*temp.*k2;
307     temp2( 3001 )
308
309
310
311 %% Clean-up
312
313 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
314     monitors = get( 0, 'MonitorPositions' );
315     if ( size( monitors, 1 ) == 1 )
316         autoArrangeFigures( 3, 4, 1 );
317     elseif ( 1 < size( monitors, 1 ) )
318         autoArrangeFigures( 2, 2, 2 );
319     end
320 end
321
322 fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );
323
324
325
326 %% Reference(s)

```

4 Appendix - Matlab Code for Problem 4

```
1
2
3
4 %% Synopsis
5
6 % Problem 4 – Washing Machine Abuse Testing
7
8
9
10 %% Environment
11
12 close all; clear; clc;
13 % restoredefaultpath;
14
15 addpath( genpath( './00 Support' ), '-begin' );
16
17 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
18 set( 0, 'DefaultFigureWindowSize', 'normal' );
19 set( 0, 'DefaultLineLineWidth', 0.8 );
20 set( 0, 'DefaultTextInterpreter', 'Latex' );
21
22 format ShortG;
23
24 pause( 1 );
25
26
27
28 %% Impulse Signal
29
30 time_step = 1e-4; % s
31 net_time = 10; % s
32 time_indices = 0:time_step:( net_time - time_step );
33
34 impulse = zeros( size( time_indices ) );
35 impulse( 4/time_step ) = 1./time_step;
36 brick_impulse = 5 .* impulse;
37 % brick_impulse = 50 .* impulse;
38
39
40 % figure( 'Name', 'Unit Impulse Signal' ); ... % Figure 1
41 %     plot( time_indices, impulse ); grid on;
42 %     legend( 'Unit Impulse Signal', 'Location', 'NorthEast' );
43 %     xlabel( 'Time [s]' ); ylabel( 'Force [N]' );
44 %     axis( [ 0 net_time 0 125 ] );
45
46 % return
47
48 %% 1 DOF System Impulse Function
49
50 m = 5; % kg
51
52 % Initial conditions of 1 DOF system.
53 xo = 0; % m
54 vo = 1 / m; % m/s
55
56 wd = 31.4; % radians\s or 5 Hz or 300 RPM
57
58
59 ks = 9810; % N/m
60 wo = sqrt( ks / 110 ); % 9.4 radians\s
61 fo = wo / (2*pi); % 1.5 Hz
62
63
64 epsilon = 0.25;
65 C = epsilon * 2*sqrt( ks * 110 ); % 519.4 kg\s
66
67
68 h_unit_impulse = @( washing_machine_mass, wd, wo, epsilon, t ) ( 1./( washing_machine_mass.*wd)
69 ) .* exp( -wo.*epsilon.*t ) .* sin( wd.*t ); % mvo = 1 kg\ (m s)
70
71
72 %% 1 DOF System Impulse Response
73
74 impulse_response = h_unit_impulse( 100, wd, wo, epsilon, time_indices );
```

```

75 %
76 figure( 'Name', '1 DOF System Impulse Response' ); ... % Figure 2
77 plot( time_indices, impulse_response ); grid on;
78 xlabel( 'Time [s]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
79 axis( [ -0.1 3 -2.5e-4 3.0e-4 ] );
80
81
82 response_to_brick_perturbation = conv( brick_impulse, impulse_response ) * time_step;
83 time_indices_perturbation = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
    time_step;
84 %
85 figure( 'Name', '1 DOF Impulse Response' ); ... % Figure 3
86 plot( time_indices_perturbation, response_to_brick_perturbation ); grid on;
87 xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
88 axis( [ -0.1 5 -1.5e-3 1.5e-3 ] );
89
90
91
92 %% Sinusoidal Impulse Response
93
94 % sinusoid_input = 493.5 * sin( 2 * pi * 5 * time_indices );
95 sinusoid_input = 1 * sin( 2 * pi * 5 * time_indices );
96 % sinusoid_input = 493.5 * sin( 2 * pi * 1 * time_indices );
97 sinusoid_input( 1:1:( 1 / time_step ) ) = 0;
98 sinusoid_input( ( 8 / time_step ):1:( 10 / time_step ) ) = 0;
99 %
100 figure( 'Name', 'Sinusoidal Input' ); ... % Figure 4
101 plot( time_indices, sinusoid_input ); grid on;
102 xlabel( 'Time [s]' ); ylabel( 'Force [N]' );
103 % xlim( [ 0 2.5 ] );
104 ylim( [ -600 600 ] );
105
106
107 response_to_sinusoidal_forcing = conv( sinusoid_input, impulse_response ) * time_step;
108 time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_sinusoidal_forcing ) - 1 ) ) .*
    time_step;
109 %
110 figure( 'Name', '1 DOF Sinusoidal Forcing Response' ); ... % Figure 5
111 plot( time_indices_sinusoidal_forcing, response_to_sinusoidal_forcing ); grid on;
112 xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
113 % xlim( [ 0 2.5 ] );
114
115
116
117 %% Combined Response
118
119 joint_signal = sinusoid_input + brick_impulse;
120 %
121 figure( 'Name', 'Combined Signal' ); ... % Figure 6
122 plot( time_indices, joint_signal ); hold on;
123 plot( time_indices, sinusoid_input, 'LineStyle', '—' );
124 plot( time_indices, impulse, 'LineStyle', '—' );
125 xlabel( 'Time [s]' ); ylabel( 'Force [N]' );
126 % xlim( [ 0 2.5 ] );
127 % ylim( [ -600 600 ] );
128
129
130 response_to_combined_forcing = conv( impulse_response, joint_signal ) * time_step;
131 time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
    time_step;
132 %
133 figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
134 plot( time_indices_sinusoidal_forcing, response_to_combined_forcing ); grid on;
135 xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
136 % xlim( [ 0 2.5 ] );
137
138
139
140 %% Plot Difference
141
142 figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
143 plot( time_indices_sinusoidal_forcing, response_to_combined_forcing -
    response_to_sinusoidal_forcing ); grid on;
144 xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
145 % xlim( [ 0 2.5 ] );
146
147
148

```

```
149 %% Clean-up
150
151 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
152     monitors = get( 0, 'MonitorPositions' );
153     if ( size( monitors, 1 ) == 1 )
154         autoArrangeFigures( 3, 4, 1 );
155     elseif ( 1 < size( monitors, 1 ) )
156         autoArrangeFigures( 2, 2, 2 );
157     end
158 end
159
160 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
161
162
163
164 %% Reference(s)
```
